

TET technical paper

Version 2 · testnet vo.2, as the code stands · 2026-10-10 · written against commit [504a4608cob2249dd9aa92d16d54ee2ed05581b2](#) · the public seeds run bea1e45 · this is a testnet; data may be reset

Your proofs and keys, in your hands, not a company's. TET is a public test network for checking who signed something, and when: every transaction and signed record carries two signatures, Ed25519 and ML-DSA-44; members can post anonymously with a RISC Zero membership proof; messages and files are end-to-end encrypted with X25519 and CRYSTALS-Kyber-768 (Round 3), with keys and senders checked in the browser. This paper describes what the code does at the commit named above, including what it does not protect; Sections 11 and 13 are the parts about plans.

o. Version and changelog

Version 2.0, dated 2026-10-10, written against the commit named at the top; scope: the public test network, vo.2. Where this commit has code the public seeds don't run yet, the text says so.

This paper marks itself: its HTML and its PDF are each signed by TET's publisher ID, and each mark has a proof code. Every earlier version stays published with its codes, so the paper's own history can be checked, including with the offline verifier (Section 10).

Version	Date	Text (HTML)	PDF	Status
1	2026-10-09	TET-418B-CFT2	TET-QQGQ-B5MM	superseded by 2 (files kept: tet-technical-paper-v1.*)
2	2026-10-10	this version's code, beside the download	this version's code, beside the download	current

Sources: [tet-network/ui/app/whitepaper/marks.json](#), [tet-network/ui/scripts/paper_build.mjs](#)

1. A public proof layer no company owns

Your proofs and keys, in your hands, not a company's.

Why now: generated text, images and voices are cheap, so being able to check who signed what, and when, without trusting a company matters more than whether something looks real.

The honest limit: lose your passphrase (12 words) and nobody can recover it; and today TET still runs on one operator and one block producer (Sections 7 and 13).

What TET is for: proofs of who, when and what that anyone can check without trusting a company, and that stay checkable if TET itself disappears. What each neighbouring system is for, and what TET adds or leaves to it:

	What it's for	What TET adds	What TET leaves to it
Bitcoin	Money; a very hard-to-rewrite ledger	Signed records of who, when and what, with hybrid post-quantum signatures	Money, and timestamps far harder to rewrite than TET's today
Ethereum	Programs that run on a chain	Records, membership proofs and messaging, not programs	Smart contracts
C2PA	Credentials embedded by cameras and editing tools	Marks a person chooses to make on any file's hash, checkable offline	Provenance from the capture device itself
OpenTimestamps	Free timestamps anchored in Bitcoin	Who signed, not only when; anonymous membership	Stronger timestamps (Bitcoin-anchored)
Passkeys	Phishing-resistant sign-in	Per-site keys from words you hold; a membership-only mode (design, Section 11)	Platform sync and recovery
Signal	Mature end-to-end messaging	Messaging tied to the same keys as records; safety numbers	Metadata protection, and a mature, audited protocol
Tor	Hiding where network traffic comes from	Hiding which member posted, among members	Hiding the network address: TET doesn't

Sources: tet-network/ui/app/try/WhatPanel.tsx, [docs/plans/PAPER_V2_OUTLINE.md](#)

2. Principles

- Verify, don't trust: every signature and proof is checked by the reader's own page or tool, not taken from a node.
- Your keys and proofs in your hands: keys come from words the person holds; records are files they keep.

- Every feature says what it proves and what it doesn't, next to the feature.
- Weaknesses are published: SECURITY.md lists known limitations and every fixed issue, by class.

Sources: [SECURITY.md](#), [docs/THREAT_MODEL.md](#)

3. Architecture

A node is one Rust binary, `TET-Core`. On start it reads its configuration from the environment, opens its libp2p and ML-DSA keystores, opens the ledger database, creates the genesis state if the chain is empty, restores its mempool, and starts three libp2p swarms (the block plane, the inference plane and the ledger plane), a swarm health watchdog, the block producer if `TET_AUTO_MINE` is set, and the REST server (port 5010 by default).

Storage is one sled database per node (`TET_DB_DIR`). The ledger keeps balances, blocks by height and by id, the canonical chain, undo records for reorgs, a transaction index and pending transactions in separate trees. Messages, files, the signature registry, sites and the operator's hide list live in the same database but outside the ledger: they are not part of the state root and are not on the chain.

A block (`BlockRecordV1`) records its height, block id, parent id, producer id, the transaction hashes and transactions, the state root and the reward. Its id is SHA-256 over `TET_BLOCK_ID_V2|`, the height (u64, little-endian), `|parent=`, the parent id, `|state=`, the state root, `|txs=`, the comma-joined transaction hashes and `|producer=`, the producer id. There is no Merkle transaction root, and the id commits to no timestamp and no signature. The state root is SHA-256 over the sorted balance rows: it covers balances only. The default block time is 10 seconds; the seeds run 12.

A transaction is a `SignedTxEnvelopeV1`: the transaction (`TxV1`, one of `transfer`, `signer_link`, `founding_member_enroll`, `genesis_bridge`, `initial_airdrop`, `file_fee`, `worker_register`, `enterprise_inference`, `verify_zk_proof`), a hybrid signature and an attestation. Its hash is SHA-256 of its chain-bound signing message (Section 4). Since commit 01d7192, a transaction must be signed by the wallet it acts for; this is checked when it is admitted, when a block is previewed and when a block is applied. The mempool holds up to 10,000 transactions or 64 MiB and evicts the lowest fee.

The block plane uses TCP with Noise and Yamux, gossipsub in strict mode with signed messages, Kademia, identify and mDNS. Gossip topics: `/tet/v1/blocks`,

`/tet/v1/txs`, `/tet/v1/tmail`, `/tet/v1/files/announce`, `/tet/v1/ai-workload`. Request-response protocols carry chain hellos, catch-up ranges (up to 100 blocks or 8 MiB), single blocks by id, anonymous-membership data and file fetches. If the swarm's event loop stalls for 180 seconds, the watchdog exits the process with status 70 so its supervisor restarts it.

In public mode (`TET_PUBLIC_API`), the REST server answers only an allow-list of routes: chain status and explorer reads, the signature registry, sites, Tmail, Files and the anonymous-membership routes. Administrative routes answer only on loopback. Each client address gets token buckets: 10 reads a second (burst 40) and 20 writes a minute (burst 10), plus a daily upload budget.

Sources: [tet-core/src/main.rs](#), [tet-core/src/ledger.rs](#), [tet-core/src/consensus.rs](#), [tet-core/src/protocol.rs](#), [tet-core/src/p2p.rs](#), [tet-core/src/sync.rs](#), [tet-core/src/swarm_health.rs](#), [tet-core/src/rest/public_api.rs](#), [tet-core/src/rest/state.rs](#)

4. Cryptography

Every signed object carries a hybrid signature: an Ed25519 signature and an ML-DSA signature over the same bytes, and both must verify. Ed25519 is `ed25519-dalek 2.2`. ML-DSA (FIPS 204) is `dilithium-rs 0.2.0`, a single-maintainer crate with no published third-party audit; the node vendors NIST's ACVP ML-DSA-44 key-generation and signature-verification vectors as tests. ML-DSA signing is deterministic: its randomness is SHA-256 of `tet:mldsa44-signing-rnd:v1` and the message.

Domain separation. Most objects sign a `|`-joined string that starts with a tag naming the object and binds the chain: for a transaction, `tet tx`

`v1|chain_id=...|genesis_hash=...|mldsa=...|tx=...`; for a Tmail envelope, `tet tmail envelope`

`v1|chain_id=...|genesis_hash=...|msg_id=...|...|payload_sha256=...|mldsa_pk=...`; for a site edit, `tet site edit v1|...`. A signature for one chain, or for one kind of object,

does not verify as another. These strings are not length-prefixed. Signature records and agent payloads use PAE (DSSE-style pre-authentication encoding, length-prefixed):

`PAE("tet agent payload v1", [chain_id, genesis_hash, payloadType, payload])`, with payload types `application/vnd.tet.sha256` (a file's hash), `tet sig publish v1` (the signer's consent to publish a record) and `tet agent manifest v1`. Since commit `bea1e45`, messaging-key registrations are PAE too, under `tet tmail key v2` (Section 6).

ML-DSA level. New keys are ML-DSA-44. Signature records, agent payloads and the demo's fee sponsor accept only ML-DSA-44 (1312-byte keys, 2420-byte signatures). Transactions, Tmail, sites and files still accept a key of level 44, 65 or 87, chosen by its length.

Wallet ID today. A wallet comes from 12 (or 24) BIP39 words. The Ed25519 secret key is the initial 32 bytes of the BIP39 seed (empty passphrase, no derivation path), and the wallet ID is the hex of the Ed25519 public key. The ML-DSA key comes from the same seed: HKDF-SHA256 with info `tet:pqc:mldsa44-seed:v1` gives a 32-byte seed for deterministic key generation. The wallet ID commits only to the Ed25519 key: the ML-DSA key is not bound to it. Binding both keys (`wallet_id_v2`) is a Phase 1 change (Section 13); until then the post-quantum half of a signature does not tie it to the wallet (SECURITY.md).

Hashes are SHA-256 throughout: transaction hashes, block ids, record hashes, site edit chains, commitments and nullifiers. A proof code `TET-XXXX-XXXX` is the leading 40 bits of a signature record's SHA-256, in Crockford base32. It is a lookup key, not a proof: the record's two signatures are what is checked.

In the browser, Ed25519 is `@noble/ed25519` and `@noble/curves`; ML-DSA is `tet-pqc-wasm`, the same `dilithium-rs` compiled to WebAssembly. A page checks records and envelopes itself: it rebuilds the signed bytes, pins ML-DSA-44 sizes and verifies both signatures, against the chain id and genesis hash it reads from the node (or that the reader types in). Tests check that the node, the WebAssembly module and the browser derive the same keys from the same words.

Sources: [tet-core/src/wallet.rs](#), [tet-core/src/quantum_shield.rs](#), [tet-core/src/fips204_vectors.rs](#), [tet-core/src/agent.rs](#), [tet-core/src/sigs.rs](#), [tet-core/src/rest/helpers.rs](#), [tet-pqc-wasm/src/lib.rs](#), [tet-network/ui/app/lib/proof_code.ts](#), [tet-network/ui/app/lib/ed25519_tet.ts](#), [SECURITY.md](#)

5. Anonymous membership

Joining. A member registers a commitment, SHA-256 of `tet-anon-v1` and a 32-byte member secret, in a record signed by their wallet. The secret is derived from the wallet's words (HKDF-SHA256, info `tet-anon-member-v1`) and never leaves the member's device. Registration is free and the registry is node-local, off the chain. Every 60 seconds the node starts a new epoch; a registration counts from the next epoch. The members form a Merkle tree of depth 20 (up to 1,048,576 leaves), ordered by wallet id, with parent `SHA-256("tet-node-v1" || left || right)`.

The proof. A RISC Zero zkVM guest (risco-zkvm 3.0.5) proves, using SHA-256 only, that the prover knows a secret whose commitment is a leaf under a given root. Its public output (journal, kind TAM1) is the root, a nullifier, a one-time Ed25519 key that signs the post, the receiver (a board or a person) and the UTC day. The nullifier is `SHA-256("tet-null-v1" || secret || receiver || day)`, with the day as a little-endian u64 of `sent_at_ms / 86,400,000`. The prover runs on the member's own machine (127.0.0.1:9945), because it receives the secret.

Checks on the node. The envelope's one-time key must be the journal's, and its receiver and day must match. The receipt (about 250 KiB, fetched separately from gossip) must hash to the announced value, come from TET's pinned membership program (`ANON_GUEST_IMAGE_IDS`, or an id the operator adds), and verify. The root must be one the node built for an epoch within the last hour, for a day within one of today. The nullifier is then claimed: one message per nullifier. Since commit f5f968a the node does all of this before it stores or relays a post; a post that fails is refused, and one that arrived another way is deleted.

What follows. One member has one daily ID per board per UTC day (up to three around midnight UTC, because a day either side is accepted). A post's daily ID is the opening four hex digits of its nullifier, shown only once the proof is verified: the same for one member on one board for one day, different on the next. Members-only polls use a member list chosen by the poll's creator (3 to 1,000 wallets); the node builds that list's root from its own registry, so a poll can't be filled with invented members.

Fast posting (at this commit; not yet on the seeds). A post whose proof verifies registers its one-time key for that board and UTC day; later posts that day by the same member carry no proof and are signed with that key, so they appear at once. The day's opening post still takes about 30 seconds to prove. The node holds unregistered gossiped posts only while their registering post is checked, keeps at most 20 posts per daily ID on a board, paces each key (five at once, then one every 3 seconds, 200 a day), and never takes a post without a proof for a poll (one proof, one ballot).

Limits. A member is anonymous among the registrations their node has seen; on today's testnet that set is small, and so is the anonymity. Registration is free, so the set has no sybil resistance yet. The node sees the poster's IP address and timing.

Sources: [methods/guest/src/main.rs](#), [nexus-protocol/src/lib.rs](#), [prover/host/src/main.rs](#), [tet-core/src/tmail/anon.rs](#), [tet-core/src/tmail/store.rs](#), [tet-core/src/tmail/poll.rs](#), [tet-core/src/tmail/envelope.rs](#), [tet-network/ui/app/lib/anon_tree.mjs](#), [tet-network/ui/app/lib/board.mjs](#), [docs/plans/FAST ANON POSTING.md](#)

6. Messaging: Tmail and Files

Key exchange is X25519 plus CRYSTALS-Kyber-768 Round 3, which is not FIPS 203 ML-KEM: the two are byte-incompatible. The node uses `pqcrypto-kyber 0.8.1` and the browser `crystals-kyber-js 1.1.2` (with `x25519-dalek 2` and `@noble/curves`). Fields named `mlkem_*` are legacy names for these Kyber Round 3 keys. Moving both planes to FIPS 203 ML-KEM-768 is a Phase 1 item.

Encryption. For each message the sender makes a fresh X25519 key and a fresh Kyber encapsulation to the receiver's keys; HKDF-SHA256 over both shared secrets, with info `tet-e2ee-hybrid-v1`, gives a 32-byte key for ChaCha20-Poly1305 with a 12-byte nonce. Messaging keys come from the wallet's words (HKDF labels `tet-tmail-x25519-v1` and `tet-tmail-mlkem-v1`) and are published in a registration signed by the wallet over PAE `tet tmail key v2`; the node takes nothing else.

Checked in the browser (since commit `bea1e45`, deployed 2026-10-10). Before encrypting a message or a file, the page fetches the recipient's registration and checks that both signatures are the recipient wallet's; an older or unsigned registration is refused with "ask them to open TET once to re-register". Before showing who sent something, the page checks the sender's signature; a forged one isn't shown. Each conversation shows a safety number, 20 digits from both people's keys: the same number on both screens means nobody in between replaced a key.

The envelope is signed by the sender's wallet, or, for an anonymous post, by the one-time key in its proof. The signature covers the message id, flags, sender, receiver, release time, fee and the SHA-256 of the ciphertext. Burn after reading is a signed request, by the sender or the receiver, that cooperating nodes delete the message: best effort, not cryptographic erasure. A time lock is a signed release time before which cooperating nodes withhold the ciphertext; anyone holding the receiver's keys and the ciphertext could read it earlier.

Retention. Messages are kept 7 days by default and 30 at most; a named sender's newest 5 per conversation, a receiver's newest 100 anonymous messages (all of a poll's ballots), and 50,000 messages in all. Files use the same key exchange (HKDF info `tet-file-v1`), with the name, type and body encrypted separately; the code's defaults are 5 MB a file and 30 days, and the demo node is configured for 100 MB and 7 days.

What a node sees: the receiver, the sender's wallet (or `anonymous` and the one-time key), the times, the flags, the sizes and, for an anonymous post, its journal. It holds only

ciphertext and has no decryption route; only the two people can read a message as long as their safety numbers match and the page they use is the honest one.

Sources: [tet-core/src/e2ee.rs](#), [tet-core/src/tmail/envelope.rs](#), [tet-core/src/tmail/keys.rs](#), [tet-core/src/tmail/burn.rs](#), [tet-core/src/tmail/timelock.rs](#), [tet-core/src/tmail/store.rs](#), [tet-core/src/files/mod.rs](#), [tet-core/src/files/storage.rs](#), [tet-network/ui/app/lib/tmail_e2ee.ts](#), [tet-network/ui/app/lib/tmail_keys.ts](#), [tet-network/ui/app/lib/files_e2ee.ts](#), [tet-network/ui/app/lib/key_trust.ts](#), [tet-network/ui/package.json](#)

7. Consensus today

TET today has one block producer. The Helsinki seed is the only validator, by explicit configuration (`TET_VALIDATOR_IDS`); the other seed and any other node follow it. The code has leader election for several validators, but with one validator the leader is always the same node. One producer can censor, reorder or delay transactions, and decides when blocks happen.

A follower applies a block only if the height continues its chain, the producer is in its validator set and is the expected leader, the parent matches, no transaction repeats, the block id recomputes, the reward is right, and the state root matches both before and after applying. Everything in that list can be recomputed by a follower, so a producer can't make a follower accept an invalid balance change.

Blocks carry no producer signature yet, and the producer id in a block is an unsigned string. What authenticates a block's source today is the producer's libp2p identity: followers pin the producer's PeerId (`TET_PRODUCER_PEERS`). For gossip, `gossipsub` in strict mode authenticates the message author; for sync (catch-up ranges and blocks fetched by id), a follower takes blocks only from trusted peers, its pinned producer or relays its operator lists. The sole producer takes blocks from no peer unless its operator names one. Producer signatures arrive in Phase 1 (Section 13).

Forks resolve to strictly greater cumulative weight; otherwise the local chain is kept. The block time is a timer on the producer's clock, and vesting locks compare against the local clock when a block is applied: wall-clock time is a consensus input (`SECURITY.md`).

Sources: [tet-core/src/consensus.rs](#), [tet-core/src/p2p.rs](#), [tet-core/src/sync.rs](#), [tet-core/src/ledger.rs](#), [SECURITY.md](#)

8. Threat model

The full model is docs/THREAT_MODEL.md; condensed:

Acting for someone else's wallet: a transaction must be signed by the wallet it acts for. Today that means its Ed25519 key, because the wallet ID is the Ed25519 public key and the ML-DSA key isn't bound to it; once `wallet_id_v2` binds both keys (Phase 1), forging a transaction will require breaking both Ed25519 and ML-DSA-44.

Area	Status	Whose job	How
Network (public Wi-Fi, DNS)	Partly	TET (TLS), the user	HTTPS with HSTS; signatures and messaging keys checked in the browser
Managed device (school, employer)	Out of scope	The device's owner	The admin can see everything; the page says so
DDoS	Partly	TET operator	Per-address limits; one operator can be overwhelmed
Device malware, keyloggers, extensions	Out of scope	The user's device	Keys stored only encrypted, auto-lock for a remembered ID
Served page replaced	Partly: the largest gap	TET operator; later an extension or app	A CSP admitting only this origin's scripts, no third-party code, SRI where Next.js emits it (part of the chunks today); planned: a publisher-signed build manifest
A node serving wrong keys	Defended	TET (page code)	Registrations and senders verified in the browser; safety numbers
Phishing, seed-phrase scams	Partly	TET (wording, origin binding), the user	Fixed sentences: the words are asked for only on the restore screen; sign-in bound to origin (design)
Replay, cross-protocol reuse	Defended	TET	Chain-bound PAE domains per kind of signature; single-use challenges (design)
Quantum	Partly	TET (Phase 1)	Hybrid signatures; Kyber Round 3 key exchange; the wallet id binds only Ed25519 until <code>wallet_id_v2</code>
Supply chain	Partly	TET maintainers	Lockfiles, installs without scripts, Actions pinned by SHA, code owners

Area	Status	Whose job	How
Lost words	Out of scope by design	The user	Nobody can recover them; said on every screen that uses them

Design rules, as of this commit: origin-bound single-use challenges, per-site keys and a separate `tet signin v1` domain (Sign in with TET, designed, not built); plain-words previews and no SMS, email or push approvals; a CSP and no third-party code, with SRI only partial; keys in memory, an encrypted vault with a strength check and auto-lock; three fixed sentences; supply-chain pinning; outside text treated as data; a `security.txt`. What no model or test covers is reached through the Phase 2 external audit and private reports (SECURITY.md).

Sources: [docs/THREAT_MODEL.md](#), [tet-network/ui/next.config.ts](#), [tet-network/ui/app/lib/device_store.ts](#), [tet-network/ui/app/try/ContinueBlock.tsx](#)

9. Known weaknesses and security history

SECURITY.md is the public list, kept with the code. Known limitations today include: no audit; a young signature stack; two seeds and one producer; a producer that has stalled before (for 33 hours, with a postmortem); identity binding that rests on Ed25519 alone; blocks authenticated by PeerId rather than a producer signature; weak anonymity by construction; some balance writes outside consensus; wall-clock time as a consensus input; invalid zero-knowledge receipts refused but not punished; and a development chain id on the testnet.

Fixed issues, by class (details are kept in a private repository):

- The desktop's anonymous mode could send a named message (fixed before PR #17).
- A transaction did not have to be signed by the wallet it acts for (fixed and deployed 2026-10-08; the chain had no such transaction).
- An anonymous post whose proof did not verify could still be kept and passed on (fixed and deployed 2026-10-09).
- Blocks received during chain sync were not held to the producer pin (fixed and deployed 2026-10-09).
- Unverified anonymous posts could push a verified one out of a board (fixed and deployed 2026-10-09).
- Pages trusted the messaging keys a node served (fixed and deployed 2026-10-10).

To report a vulnerability, follow SECURITY.md: privately, not in a public issue.

Sources: [SECURITY.md](#), [docs/postmortems/2026-10-04-producer-wedge-33h.md](#)

10. Verify without TET

A standalone verifier, one HTML file and a one-file command-line tool, checks a record on the reader's own device with no request to any server (at this commit; published with the demo). It inlines the verifying code and TET's ML-DSA WebAssembly; its content security policy forbids every connection, and a test runs it with networking disabled.

Level 1, offline and with no chain: the two keys in a record signed this SHA-256 for this chain, and, given the file, the file hashes to exactly that. It shows the proof code and whether the signer is TET's publisher ID. It doesn't prove who holds the keys, that the content is true, or when it was signed.

Level 2, planned: inclusion at block N in a copy of the chain. Until blocks carry producer signatures (Phase 1), that can only check that a copy is consistent with itself; the reader compares its tip with independent sources.

The verifier's two files are themselves marked by the publisher ID, with their SHA-256 published beside them.

Sources: [tet-network/ui/app/lib/offline_verify.mjs](#), [tet-network/ui/scripts/build_offline_verifier.mjs](#), [tet-network/ui/public/verify/tet-verify.html](#)

11. Sign in with TET (designed, not built)

A second factor any site can add, and a membership-only mode. Design only at this commit.

- A challenge carries the requesting origin, a nonce and an expiry; it is single use, and the signer refuses unless the origin it returns to is the challenge's origin.
- Per-site keys derived from the words and the origin, so two sites can't link one person by key.
- Signatures under a separate PAE domain, `tet signin v1`, so a sign-in signature is never a valid transaction or record, and the other way round.
- The membership-only mode reveals only "a member of group X signed in"; a stable per-site pseudonym is an option the site must state.

- No passwords or password hashes on TET or any node; no SMS, email codes or push approvals; a plain-words preview before every signature; a cooldown on repeated requests.
- The limit, on every sign-in screen: if you lose your passphrase (12 words), nobody can recover your account.

Sources: [docs/plans/SIGN_IN_WITH_TET.md](#)

12. Shelter and human spaces

Shelter (at this commit; not yet on the seeds) is a members-only space: members join by an in-person invite or vouch, posts are end-to-end encrypted to members, reads need a member's signed request, and posts are never passed to other nodes. Members post under a nickname shown with a member number nobody can choose, or anonymously against Shelter's own member set.

What it proves: a vouched member wrote each post, and the space isn't served to crawlers. What it doesn't prove: that no AI was used. The house rule, "Don't post AI-written text here", is a promise between members, not a filter.

Sources: [docs/plans/SHELTER.md](#), [tet-core/src/tmail/shelter.rs](#)

13. Roadmap

Phase 0 is today's testnet. Phase 1 is a new genesis, targeted for Q1 2027: a target published as a quarter, not a promise; if it moves, it moves in public. None of the Phase 1 items runs on the testnet today.

Change	What it does	State
V3 block header	The producer's timestamp is covered by a length-prefixed (PAE) block id.	Built on the Phase 1 branch
Producer signatures	A dedicated hybrid producer key signs each block id; validators are named in the genesis.	Built on the Phase 1 branch
Per-plane keys	Separate libp2p identities for the block, inference and ledger planes.	Built on the Phase 1 branch
<code>wallet_id_v2</code>	The wallet ID commits to both keys; changes every wallet id.	Designed, not built

Change	What it does	State
Move to FIPS 203 ML-KEM-768	Both encryption planes move from Kyber Round 3 to the final standard.	Open

Phases 2 to 10 are a vision, without dates. Two of them are prerequisites for Section 1 to be fully true: Phase 2, an external security audit, and Phase 3, more than one block producer. Until then quantum resistance is incomplete until `wallet_id_v2`, and TET runs on one operator.

Sources: [docs/PHASE 1 GENESIS SPEC.md](#), [docs/QUEUE.md](#), [tet-network/ui/app/try/WhatPanel.tsx](#)

14. What TET is not

- Not money: the testnet's units are practice units that can't be exchanged for money.
- Not a smart-contract platform.
- Not IP privacy: nodes see addresses and timing; use Tor or your own node.
- Not a company, and not yet decentralized: one operator and one block producer today.

Sources: [SECURITY.md](#), [tet-network/ui/app/try/WhatPanel.tsx](#)

This paper's proof code

TET-ZSK3-VQ1J



The code marks the text of this paper: the file `paper/tet-technical-paper.html`,
SHA-256

`54461c77ae03c170d9ef53e0b5f84fec849eb0e84b31a571d486d2e679d70324`,
marked by TET's publisher ID `e93a3597f03a8ee4621d8c7340d37fe871875f8dedbc5e`
`c9e139293d0af847bd`.

Check it: enter the code at try.stevenexus.org, or scan the QR. Or drop that HTML file
into the search box: an exact copy matches; a copy changed by one byte doesn't.

What it proves: this ID marked exactly that text. What it doesn't prove: who wrote it, or that it is
correct. This PDF file has a code of its own, shown next to its download link (a file can't contain its own
code).